

```
# FuelSync – Complete Functions & Backend Guide
### For Presentation Use – Daniel Markman
```

```
---
```

HOW THE APP WORKS (Big Picture)

FuelSync is a **local-first PWA** (Progressive Web App). That means:

- All your food data is saved **directly in your browser** (called IndexedDB – like a tiny database inside Chrome/Safari)
- There is **no traditional server** storing your data
- The only "server" calls go to **Cloudflare Workers** – lightweight serverless functions that proxy AI (Gemini) and Strava
- The app works **offline** – your data never disappears if the internet is down

```
---
```

PART 1 – DATABASE (lib/db.ts)

This file defines the structure of the local database using **Dexie.js** (a wrapper around IndexedDB).

Tables (like Excel sheets stored in your browser)

`profile` table – stores ONE row: your personal stats

- Fields: display name, weight, height, age, gender, activity level, daily calorie goal
- Also stores Strava tokens (so Strava stays connected)

`food\_logs` table – stores every food you've ever logged

- Fields: food name, calories, protein, carbs, fat, weight in grams, meal type (breakfast/lunch/etc), timestamp, date
- Also stores 7 micronutrients: fiber, cholesterol, sodium, vitamin C, vitamin D, calcium, iron
- Has a ``removed`` flag for soft-delete (food appears "removed" but isn't fully deleted)
- Has a ``sync_id`` (UUID) – a unique ID for syncing across devices without duplicates

`pin\_state` table – stores your security PIN

- Fields: hashed PIN, salt (random bytes for security), attempt counter, lockout timer
- Also stores security question/answer hashes for PIN recovery

`weight\_logs` table – body weight history over time

The Database Class (FuelSyncDB)

This is the main database class. It defines 3 versions of the schema – each version is a migration, meaning the database structure can be upgraded without losing data. Version 1 = launch, Version 2 = added `sync_id`, Version 3 = added weight logs.

```
---
```

PART 2 – FOOD LOGGING (api/localFood.ts)

This is the heart of the app. Every food log goes through these functions.

``toFoodLog(row)`` – private converter

What it does: Converts a raw database row into a clean ``FoodLog`` object the UI can use.

Why it exists: The database stores things in one format; the UI needs them in another. This bridges that gap. It also handles the ``sync_id`` – uses the UUID as the ID if present, otherwise falls back to the numeric database ID.

```

#### `upsertD1Logs(d1logs)` – private sync helper
**What it does:** Takes food logs downloaded from the cloud (Cloudflare D1 database) and merges them into local IndexedDB without creating duplicates.
**How it works:** For each cloud log, it checks if it already exists by `sync_id`. If it's a legacy log (ID is just a number), it backfills the `sync_id`. If it's genuinely new, it inserts it.
**Why it matters:** This is what makes cross-device sync work – you can log on your phone and see it on your laptop.

#### `pullWithKey(key, fetcher)` – private cache manager
**What it does:** Downloads data from the cloud, but with a smart cache – it won't download the same data more than once every 30 seconds.
**Also:** If two parts of the app try to download the same data at the same time, it deduplicates the request (only one actual network call happens).
**The cache key:** `d:2026-05-19` for a specific date, `all` for everything.

#### `withTimeout(promise, ms)` – private timeout wrapper
**What it does:** Puts a time limit on any async operation. If the cloud sync takes more than 3 seconds, the app just uses local data instead of waiting.
**Why it matters:** The app must never feel slow because of the internet. Local data always loads instantly.

#### `clearPullCache()` – exported
**What it does:** Clears the 30-second cache so the next `getLogs` call will always re-download from the cloud.
**When it's used:** After you log a new food, so the cloud data is fresh.

#### `getLogs(date)` – main function for today's food list
**What it does:** Returns all food logs for a given date (e.g. "2026-05-19").
**How it works:**
1. Tries to pull cloud data (with a 3-second timeout)
2. Queries IndexedDB for that date
3. Filters out soft-deleted entries (removed = true)
4. Converts each row using `toFoodLog`
**Used by:** HomeScreen (for macro totals) and FoodScreen (for the food list)

#### `getAllLogs()` – full history
**What it does:** Returns every food log ever, sorted newest-first.
**Used by:** HistoryScreen

#### `addLog(entry)` – log a new food
**What it does:** Saves a new food entry to IndexedDB.
**How it works:**
1. Generates a UUID (`sync_id`) for cross-device dedup
2. Determines the date from `logged_at` (or uses now)
3. Writes all fields to IndexedDB (including 7 micronutrients)
4. Immediately queues a background sync to the cloud (fires-and-forgets – app doesn't wait)
5. Returns the saved `FoodLog` object
**What it accepts:** food name, calories, protein, carbs, fat, weight, meal type, optional image URL, optional ingredients list, optional micronutrients

#### `deleteLog(id)` – permanently delete a food log
**What it does:** Hard-deletes a food log from IndexedDB and queues cloud deletion.
**Handles two ID formats:** UUID (new logs) or numeric string (legacy logs)

#### `softDeleteLog(id)` – mark food as removed (undo-able)
**What it does:** Sets `removed = true` on a log – it disappears from the active food list but stays in the database.
**Why:** So you can undo removing a food within the same session. The History screen still shows it as "removed."

```

```

### `unremoveLog(id)` – undo a soft-delete
**What it does:** Sets `removed = false` to restore a food log.
**Smart duplicate check:** Before restoring, it checks if an identical active entry already exists for the same day (can happen from old sync bugs). If a duplicate exists, it deletes the removed copy instead of creating two entries. Considers two foods identical if: same name + calories within  $\pm 5$ kcal.

### `estimateByWeight(food_name, weight_grams)` – AI macro estimate
**What it does:** Sends a food name + weight to the AI Worker and gets back estimated macros.
**Example:** "chicken breast", 200g → AI returns protein, carbs, fat, calories, micronutrients
**Used in:** Manual mode when you type a weight

### `estimateByDescription(description)` – AI free-text analysis
**What it does:** Sends a plain text description to the AI Worker.
**Example:** "bowl of pasta with tomato sauce and mozzarella, about 400g" → full macro breakdown
**Used in:** AI Smart mode in FoodScreen

### `suggestMeal(context, size)` – AI meal recommendation
**What it does:** Asks AI to suggest a real, specific meal for a given context.
**Context examples:** "pre_workout", "morning", "rest day evening"
**Size:** "big" = full meal, "small" = light snack
**Returns:** A suggested meal name with full macros + ingredient breakdown

### `estimateSteps(description)` – AI step count estimator
**What it does:** Describes your day to AI and gets an estimated step count.
**Example:** "I work at a desk, went for a 30 minute walk at lunch" → ~8,500 steps, label = "normal"
**Labels:** low (<6,000), normal (6,000–10,000), high (>10,000)

### `analyzeByImage(file)` – AI photo food analysis
**What it does:** Takes a photo of food, converts it to base64, sends to AI Worker, returns macro estimate.
**How it works:** Uses the browser's FileReader API to convert the image file to base64 string, then sends it alongside the prompt to Gemini's vision model.

---

## PART 3 – USER PROFILE (api/auth.ts)

These functions manage your body stats (weight, height, age, etc.). There is no server – everything goes to IndexedDB.

### `toBackendUser(p)` – private converter
**What it does:** Converts a raw database `LocalProfile` row into a `BackendUser` object with clean field names.
**Why the name "BackendUser"?** Historical – when the app had a real backend, this was the shape the server returned. The name is kept for compatibility with the rest of the codebase.

### `getProfile()` – read your profile
**What it does:** Reads the profile row from IndexedDB and returns it as a `BackendUser`, or `null` if no profile exists yet.
**When null is returned:** First launch before onboarding is complete.

### `createProfile(data)` – save profile for the first time
**What it does:** Clears any existing profile (safety measure) and saves a new one.
**Used by:** Onboarding screen (first-time setup)

### `updateProfile(data)` – edit your profile
**What it does:** Finds the existing profile and applies partial updates (only

```

the fields you pass change).

Example: `updateProfile({ weightKg: 78 })` – only weight changes, everything else stays.

Throws an error if no profile exists.

clearProfile() – wipe profile

What it does: Deletes the entire profile table. Used during sign-out / data wipe.

PART 4 – NETWORK CLIENT (api/client.ts)

This is the communication layer between the app and Cloudflare Workers.

workerPost<T>(worker, path, body) – send a POST request to a Worker

What it does: Sends a JSON POST request to either the AI Worker or Strava Worker, and returns the parsed response.

Type parameter <T>: TypeScript generic – you tell it what shape to expect back (e.g. `AIEstimate`).

Error handling: If the response is not OK, throws an Error with the message from the worker.

Two workers available: `'ai'` → `VITE_AI_WORKER_URL`, `'strava'` → `VITE_STRAVA_WORKER_URL`

workerGet<T>(worker, path) – send a GET request to a Worker

What it does: Same as `workerPost` but for GET requests (no body).

Used by: `getStravaAuthUrl` to get the Strava authorization link.

PART 5 – STRAVA INTEGRATION (api/strava.ts)

Connects to Strava to pull your running stats.

getProfile() – private helper

What it does: Reads the first row from the profile table. Used internally to get Strava tokens.

updateStravaTokens(tokens) – private helper

What it does: Saves updated Strava access/refresh tokens to the profile in IndexedDB.

When called: After OAuth connects, and after a token refresh.

refreshIfNeeded() – private auto-refresh

What it does: Checks if the Strava access token is about to expire (within 60 seconds of expiry). If so, calls the Strava Worker to get a new token, saves it, and returns the fresh token.

Why this exists: Strava access tokens expire every 6 hours. Without this, Strava would disconnect every day.

getStravaAuthUrl(platform) – get OAuth login link

What it does: Asks the Strava Worker for a Strava authorization URL to show the user.

Platform: `'web'` for browser, `'android'` for native app (different redirect URI behavior)

connectStrava(tokens) – save Strava connection after OAuth

What it does: Called by `App.tsx` after the Strava Worker redirects back with tokens in the URL. Saves all tokens and athlete info (name, profile picture) to IndexedDB.

Also: Syncs tokens to the cloud in the background.

getStravaStats() – fetch running statistics

****What it does:**** Gets your Strava data – recent runs, weekly totals, year-to-date stats, all-time stats.

****How it works:****

1. Calls ``refreshIfNeeded`` to ensure token is valid
2. If no token → returns `{ connected: false }`
3. Posts to the Strava Worker with the access token
4. Worker fetches from Strava API and returns formatted data

``disconnectStrava()`` – unlink Strava

****What it does:**** Removes all Strava tokens from the profile in IndexedDB. Returns `{ connected: false }`.

PART 6 – FOOD DATABASE SEARCH (`api/openFoodFacts.ts`)

Two external food databases are used to look up nutrition by name or barcode.

``parseServingSize(s)`` – private helper

****What it does:**** Parses a serving size string like "100g" or "28.3g" into a number.

****Returns null**** if the string doesn't contain grams/ml.

``parseUSDA(f)`` – private USDA parser

****What it does:**** Converts a raw USDA API response into a clean ``OFFProduct`` object.

****USDA returns nutrients in a flat array**** – this function maps them by name into a Map for easy lookup.

****Energy handling:**** Prefers KCAL entry; if only kJ is available, converts ($\text{kJ} \div 4.184 = \text{kcal}$).

****Extracts:**** calories, protein, carbs, fat + 7 micronutrients.

``searchFood(query)`` – search by food name (USDA)

****What it does:**** Searches the USDA FoodData Central database for foods matching the query.

****API used:**** USDA FDC (``api.nal.usda.gov``) – US government free food database with ~600,000 foods.

****Returns:**** Up to 20 results as ``OFFProduct[]``, filtering out any that couldn't be parsed.

****Timeout:**** 8 seconds – app won't hang forever waiting for USDA.

****Used in:**** FoodScreen search bar

``parseOFF(raw, barcode)`` – private Open Food Facts parser

****What it does:**** Converts a raw Open Food Facts API response into a clean ``OFFProduct``.

****Handles:**** Missing product names (returns null), missing calories (returns null), both energy-kcal and energy-kJ formats.

``lookupBarcode(barcode)`` – scan a barcode

****What it does:**** Looks up a product by its barcode number (EAN-13 etc.) in the Open Food Facts database.

****API used:**** ``world.openfoodfacts.org`` – open-source crowdsourced food database.

****Returns:**** Full nutrition per 100g + serving size + product image URL, or ``null`` if not found.

****Timeout:**** 8 seconds.

PART 7 – AI CLOUDFLARE WORKER (`workers/ai/index.ts`)

This runs on Cloudflare's edge network (not your phone, not a server – Cloudflare's global CDN). It hides the Gemini API key from the browser.

`cors(origin)` – CORS headers builder
What it does: Returns the HTTP headers needed to allow the browser to talk to this Worker from a different domain (CORS = Cross-Origin Resource Sharing).
Allows: The app domain, localhost (for development), and Capacitor (for Android).

`json(data, status, origin)` – response builder
What it does: Wraps any data into a proper JSON HTTP response with CORS headers.

`err(msg, status, origin)` – error response builder
What it does: Returns an error JSON response like `{"error": "description required"}`.

`geminiWithModel(apiKey, model, prompt, maxTokens, imageBase64?, imageMime?)` – single Gemini call
What it does: Makes one API call to a specific Gemini model with a given prompt (and optionally an image).
Special handling:
- If the model name starts with `gemini-2.5-`, it sets `thinkingBudget: 0` (disables Chain-of-Thought to save tokens and speed up responses)
- If status is 429 (rate limited) or 404 (model not found) → returns a skip signal, doesn't throw
- If 401/403 → throws (invalid key, fatal error)

`gemini(keys, prompt, maxTokens, imageBase64?, imageMime?)` – smart multi-key, multi-model caller
What it does: Tries every combination of API key × Gemini model until one succeeds.
Model order: gemini-2.5-flash → gemini-2.0-flash → gemini-2.0-flash-lite → gemini-2.5-flash-lite
Key order: Key 1 → Key 2 → Key 3 (if configured)
If all combinations fail (all rate limited): throws a user-friendly error message.
Why this matters: Gemini free tier has per-model rate limits. By cascading through 4 models × 3 keys = 12 combinations, the worker almost never hits a limit.

`parseJSON(text)` – flexible JSON extractor
What it does: Extracts JSON from Gemini's response, which might not always be clean JSON.
Tries three methods in order:
1. Direct `JSON.parse(text)` – works if response is pure JSON
2. Extract from a markdown code block (`\`\`\`json ... \`\`\` `)
3. Find the first `{...}` block in the text
Throws only if all three methods fail.

`fetch` handler – main Worker entry point (routes)

The Worker receives all HTTP requests and routes them by URL path:

`POST /estimate` – macro estimate by weight
- Input: `{ food\_name, weight\_grams }`
- Prompt to Gemini: "For Xg of [food], give exact nutritional values"
- Returns: Full macro + micronutrient JSON

`POST /describe` – macro estimate from free text
- Input: `{ description }` (e.g. "grilled salmon with quinoa")
- Prompt: "Analyse this meal and provide nutritional breakdown"
- Returns: Full macro + micronutrient JSON including per-ingredient breakdown

`POST /suggest` – meal recommendation
- Input: `{ context, size }` (e.g. "pre_workout", "big")
- Prompt: "Suggest a specific, practical [size] meal for an athlete for

```

[context]"
- Returns: A real meal suggestion with macros

**`POST /analyze`** – food photo analysis
- Input: `{ base64, mimeType }` (image as base64 string)
- Sends image + prompt to Gemini Vision
- Returns: Estimated macros for the food in the photo

**`POST /steps`** – daily step estimation
- Input: `{ description }` (your day's activities)
- Uses a detailed prompt with step estimates by job type and activities
- Returns: `{ steps: number, label: "low"|"normal"|"high" }`

---

## PART 8 – STRAVA CLOUDFLARE WORKER (workers/strava/index.ts)

Handles all Strava OAuth and API proxying. Keeps `STRAVA_CLIENT_SECRET` hidden
from the browser.

### `cors`, `json`, `err` – same utility functions as AI Worker

### `fetch` handler – routes:

**`GET /auth-url`** – get OAuth authorization link
- Builds a Strava OAuth URL with the client ID and redirect URI
- Platform-aware: `android` uses `fuelsync://strava` as the deep link, `web`
uses the app domain
- Returns: `{ url: "https://www.strava.com/oauth/authorize?..." }`

**`GET /callback`** – Strava redirects here after user approves
- Receives the auth `code` from Strava
- Exchanges the code for real access/refresh tokens (POST to Strava with client
secret)
- Extracts athlete name and profile picture
- Redirects back to the app with tokens as URL query parameters (e.g. `?
strava_access_token=...`)
- Platform-aware: Android gets redirected to `fuelsync://strava?...` (opens the
native app)

**`POST /refresh`** – exchange refresh token for new access token
- Input: `{ refresh_token }`
- Posts to Strava's token endpoint with client secret
- Returns new `access_token`, `refresh_token`, `expires_at`

**`POST /stats`** – fetch athlete data from Strava
- Input: `{ access_token }`
- Makes 3 parallel Strava API calls: `/athlete`, `/athlete/activities`,
`/athletes/{id}/stats`
- Filters activities to only runs (ignores cycling, walking, etc.)
- Formats each run: converts meters → km, seconds → `MM:SS`, calculates pace per
km
- Returns structured object with: weekly totals, YTD totals, all-time totals,
recent 6 runs

---

## PART 9 – SECURITY / PIN (lib/pin.ts)

Handles all PIN security. Uses PBKDF2 – the same algorithm banks use for
password hashing.

### `randomSalt()` – private
**What it does:** Generates 16 random bytes using the browser's cryptographic

```

random number generator.

****Why:**** A random salt ensures that even if two users have the same PIN, their hashes are different.

`saltToHex(salt)` / `hexToSalt(hex)` – private converters

****What they do:**** Convert between raw bytes (Uint8Array) and hex strings for storage in IndexedDB.

`pbkdf2Hash(text, salt)` – private, the core security function

****What it does:**** Hashes a PIN using PBKDF2-SHA256 with 100,000 iterations.

****What PBKDF2 means:**** Password-Based Key Derivation Function 2. It deliberately runs slowly (100,000 rounds of SHA-256) to make brute-force attacks impractical.

****Returns:**** A 64-character hex string (256-bit hash).

`getState()` – private

****What it does:**** Reads the current PIN state row from IndexedDB.

`setupPin(pin)` – create a new PIN

****What it does:**** Clears any existing PIN, generates a new salt, hashes the PIN, saves to IndexedDB.

****Used on:**** First-time onboarding and after a PIN reset.

`setupSecurityQuestion(question, answer)` – set a recovery question

****What it does:**** Hashes the security answer (lowercased + trimmed) with a new salt and saves it alongside the question.

****The answer is case-insensitive**** – "PARIS", "paris", and "Paris" all work.

`hasPin()` – check if a PIN exists

****What it does:**** Returns `true` if the `pin\_state` table has at least one row.

****Used by:**** App.tsx to decide whether to show onboarding or the PIN screen.

`getSecurityQuestion()` – read the stored question text

****What it does:**** Returns the security question string (e.g. "What city were you born in?") or `null`.

`verifyPin(pin)` – the main login check

****What it does:**** Checks a PIN against the stored hash and manages lockouts.

****Lockout tiers (progressive):****

- 3 wrong attempts → locked for 30 seconds

- 6 wrong attempts → locked for 5 minutes

- 10 wrong attempts → locked for 30 minutes

- 15 wrong attempts → ****complete data wipe**** (IndexedDB fully deleted)

****Returns:**** `{ ok, locked, lockedUntil, attemptsLeft, wiped }`

****On correct PIN:**** Resets all attempt counters.

`verifySecurityAnswer(answer)` – check recovery answer

****What it does:**** Same logic as `verifyPin` but for the security question answer.

****Max attempts:**** 5 – after 5 wrong answers, data is wiped.

`resetPin(newPin)` – change the PIN

****What it does:**** Generates a new salt, hashes the new PIN, updates the stored record (keeps security question intact).

`clearPin()` – delete PIN state

****What it does:**** Empties the `pin\_state` table. Used during data wipe / sign-out.

PART 10 – NUTRITION HOOK (hooks/useNutrition.ts)

This React hook connects all the nutrition logic to the UI. Any screen that needs macro data uses this.


```

#### `toUserProfile(user)` – convert BackendUser to UserProfile
**What it does:** Converts the app's user object into the shape the nutrition
engine expects.
**Returns null** if any required field (weight, height, age, gender) is missing
– prevents the engine from running with incomplete data.

#### `useNutrition()` – the main hook (returned object)
**What it does:** Sets up all nutrition state and logic in one place.

**On mount effects:**
1. Checks if `todayLog` is from a previous day → auto-resets training type
2. Checks if the week has rolled over to a new Monday → auto-starts a new weekly
load
3. Recomputes macro targets whenever the user's profile stats change

**Returns these functions and values:**

#### `logDay(trainingType, plannedWorkoutTime?, dailyActivityModifier?)` – set
today's training
**What it does:**
1. Checks the **leg fatigue gate** – if you've run too much this week, it may
block certain training types
2. Creates a `DailyLog` object for today
3. Runs the nutrition engine to compute macro targets
4. Saves to the Zustand store
**Returns:** `{ blocked, message, log }` – if blocked = true, the UI shows a
warning instead of confirming.

#### `refreshWeather()` – fetch current weather
**What it does:** Gets the device's GPS location (browser geolocation API),
fetches weather from OpenWeatherMap, evaluates if conditions affect training
(e.g. heat warning, thunderstorm alert).
**Returns:** `{ weather, alert }` or `null` if no API key or location denied.

#### `getMacroBreakdown()` – detailed macro calculation
**What it does:** Runs the nutrition engine and returns the full breakdown (not
just targets – also includes reasoning, calorie distribution, etc.).
**Used by:** Any UI element that needs to show "why" macros are set a certain
way.

#### `setActivityModifier(modifier)` – adjust daily activity
**What it does:** Updates the daily activity modifier (e.g. "very_active_day" if
you walked a lot) and immediately recomputes macro targets.
**The modifier affects** total daily calorie needs on top of the base training
type calculation.

```

PART 11 – SUMMARY TABLE

Function	File	What It Does (One Line)
----- ----- -----		
`getLogs`	localFood.ts	Get all food logs for a date
`addLog`	localFood.ts	Save a new food log to IndexedDB
`deleteLog`	localFood.ts	Permanently delete a food log
`softDeleteLog`	localFood.ts	Hide a food (undo-able)
`unremoveLog`	localFood.ts	Restore a hidden food
`estimateByWeight`	localFood.ts	AI: macros for a food by weight
`estimateByDescription`	localFood.ts	AI: macros from free text description
`suggestMeal`	localFood.ts	AI: suggest a meal for a context
`estimateSteps`	localFood.ts	AI: estimate daily steps from description
`analyzeByImage`	localFood.ts	AI: identify food from a photo

```

| `getProfile` | auth.ts | Read user profile from IndexedDB |
| `createProfile` | auth.ts | Save new user profile (onboarding) |
| `updateProfile` | auth.ts | Edit existing profile |
| `clearProfile` | auth.ts | Delete profile (sign-out) |
| `workerPost` | client.ts | Send POST to Cloudflare Worker |
| `workerGet` | client.ts | Send GET to Cloudflare Worker |
| `connectStrava` | strava.ts | Save Strava tokens after OAuth |
| `getStravaStats` | strava.ts | Fetch running stats from Strava |
| `refreshIfNeeded` | strava.ts | Auto-refresh expired Strava token |
| `disconnectStrava` | strava.ts | Remove Strava connection |
| `searchFood` | openFoodFacts.ts | Search food by name (USDA database) |
| `lookupBarcode` | openFoodFacts.ts | Look up product by barcode (OFF) |
| `gemini` | workers/ai | Call Gemini AI with key+model cascade |
| `parseJSON` | workers/ai | Extract JSON from AI response text |
| `/estimate` route | workers/ai | AI macro estimate by food+weight |
| `/describe` route | workers/ai | AI analysis of a meal description |
| `/suggest` route | workers/ai | AI meal recommendation |
| `/analyze` route | workers/ai | AI food photo analysis |
| `/steps` route | workers/ai | AI step count estimation |
| `/auth-url` route | workers/strava | Generate Strava OAuth URL |
| `/callback` route | workers/strava | Handle Strava OAuth redirect |
| `/refresh` route | workers/strava | Refresh expired Strava token |
| `/stats` route | workers/strava | Fetch athlete stats from Strava API |
| `setupPin` | pin.ts | Create a new security PIN |
| `verifyPin` | pin.ts | Check PIN + manage lockouts |
| `verifySecurityAnswer` | pin.ts | Check recovery answer |
| `resetPin` | pin.ts | Change to a new PIN |
| `setupSecurityQuestion` | pin.ts | Set recovery question/answer |
| `logDay` | useNutrition.ts | Set training type + compute macros |
| `refreshWeather` | useNutrition.ts | Fetch weather + training alert |
| `getMacroBreakdown` | useNutrition.ts | Full macro calculation details |
| `setActivityModifier` | useNutrition.ts | Adjust calorie needs for activity |

```

FuelSync – Built by Daniel Markman